



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DSL Compiler for Loan Approval

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA1407 – Compiler Design

to the award of the degree of

**BACHELOR OF TECHNOLOGY IN
ARTIFICIAL INTELLIGENCE & MACHINE LEARNING**

**Submitted by
Juhi Surin (192525148)**

**Under the Supervision of
Dr.N. Deepa**



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**DSL Compiler for Loan Approval**” has been carried out by **Juhi Surin (192525148)** under the supervision of **Dr.N.Deepa** and is submitted in partial fulfilment of the requirements for the current semester of the **B.Tech Artificial Intelligence & Machine Learning** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr.W.Deva Priya,
Professor
Department of Generative AI
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr.N.Deepa,
Professor
Department of Neural Networks
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I, **Juhi Surin (192525148)** of the **B.Tech Artificial Intelligence & Machine Learning**, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled **“DSL Compiler for Loan Approval”** is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Students with Names

Juhi Surin (192525148)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Juhi Surin [192525148]	RegulaScript design, compiler structure and loan rule processing	Designed and developed the lexical, syntax and semantic analysis components and rule-processing workflow	Tested rule parsing, validation and loan decision conditions	Contributed to Chapters 1–3 and project documentation	50%
Aniksha Evanjalin M[192521308]	Loan decision engine, rule evaluation and project documentation	Developed the intermediate code generation, optimization and loan decision evaluation workflow	Tested customer inputs, compiled rules and decision outputs	Contributed to Chapters 4–5 and project documentation	50%
Total					100%

ABSTRACT

Loan approval in banking involves evaluating customer information against multiple banking policies and eligibility conditions. Traditional manual or hard-coded approaches to loan approval can require significant effort and may lead to inconsistencies and human errors. Therefore, there is a need for a structured, accurate, transparent, and efficient approach to automate the implementation of loan approval rules.

This project, “DSL Compiler for Loan Approval,” proposes a Domain-Specific Language (DSL) called RegulaScript for defining and automating banking loan approval policies. The proposed system allows authorized users to express loan approval rules using a simple and structured language instead of manually coding individual rules. The RegulaScript rules are processed through a compiler pipeline and converted into executable logic for automated evaluation.

The methodology consists of multiple compiler phases, including lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and final code generation. Lexical analysis identifies keywords, identifiers, operators, and literals, while syntax analysis verifies the grammar and constructs a parse tree. Semantic analysis validates field names, data types, and rule conditions. The generated intermediate code is optimized by removing redundant operations before producing the final compiled rule.

The compiled rules are evaluated by the Loan Compliance and Loan Decision Engine using customer information such as age, credit score, annual income, employment status, loan type, and loan amount. The system generates one of three outcomes—Approved, Rejected, or Review Required—along with the decision reason and execution status. This provides consistent and transparent loan decisions while reducing manual effort and human errors.

The project demonstrates that compiler design concepts can be effectively applied to banking automation. RegulaScript provides a structured foundation for implementing loan policies, while the compiler and decision engine enable efficient, reliable, and consistent loan evaluation.

Keywords: Domain-Specific Language, RegulaScript, Loan Approval, Compiler Design, Loan Decision Engine, Banking Automation

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	12
2	CHAPTER 2: Literature Review	16
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	21
4	CHAPTER 4: Implementation, Testing & Results, Project Management	29
5	CHAPTER 5: Conclusion & Reflection	37
	References	40
	Appendices	43

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	System Architecture	25
a	Implementation output 1	40
b	Implementation output 2	40

LIST OF TABLES

Table No.	Table Name	Page No.
3.1	Stakeholder / User Requirements	20
3.2	Functional & Non-Functional Requirements	20
3.3	Applicable Standards / Codes	22
3.4	Design Specifications	22
3.5	Alternative Solutions & Evaluation	23
3.6	Trade-off Analysis	26
3.7	Sustainability, Ethics, Safety, Risk & Security	27
3.8	Risk Register	27
4.1	Development Resources	29
4.2	Tool / Software / Equipment Used	30
4.3	Test Plan & Verification	30
4.4	Test Specification / Verification Results	31
4.5	Expected Result Categories	32
4.6	Comparison with Existing Solutions & Achievement of Objectives	34
4.7	Suggested Milestone Timeline	36
4.8	Student Assigned Responsibility / Actual Contribution	36
4.9	Project Budget / Cost	36
E.1	Sample Customer Input Parameters	44
E.2	Decision Outputs	44
J.1	Contribution Evidence Table	46

LIST OF ABBREVIATIONS / SYMBOLS

Symbol / Abbreviation	Description	Unit
DSL	Domain-Specific Language	—
DMN	Decision Model and Notation	—
DRL	Drools Rule Language	—
SQL	Structured Query Language	—
C	C Programming Language	—
IDE	Integrated Development Environment	—
GCC	GNU Compiler Collection	—
HTML	HyperText Markup Language	—
CSS	Cascading Style Sheets	—
AWS	Amazon Web Services	—
AI	Artificial Intelligence	—
ID	Identification / Customer Identification	—
AND	Logical AND operator	—
Rule₁, Rule₂, ... Rule_n	Individual loan approval rules	—
%	Percentage	%
₹	Indian Rupee	Currency

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Loan approval is an important process in the banking sector that requires evaluating customer information against predefined banking policies and eligibility conditions. Traditional loan approval processes may involve manually checking several customer attributes and applying multiple rules, which can require considerable effort and may result in inconsistent decisions and human errors.

The proposed project, “DSL Compiler for Loan Approval,” develops a Domain-Specific Language (DSL) called RegulaScript to automate the definition and execution of loan approval rules. Instead of manually coding individual loan policies, authorized users can define rules using the structured RegulaScript language. These rules are processed through different phases of a compiler and converted into executable logic. The compiled rules are then evaluated against customer information to determine loan eligibility.

The system consists primarily of a Loan Rule Compiler and a Loan Compliance and Loan Decision Engine. The compiler performs lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and final code generation. The decision engine evaluates customer details against the compiled rules and generates an appropriate loan decision.

1.2 Need for the Project

Loan approval involves multiple customer parameters and banking rules that must be evaluated correctly and consistently. Manual implementation of such rules can increase the possibility of errors and make modifications to banking policies more difficult.

The project is needed to provide a structured mechanism for defining, validating, and executing loan approval policies. Banking personnel and organizations involved in loan processing can benefit from an automated rule-based approach.

The existing approach has limitations such as:

- Dependence on manual evaluation of loan eligibility.

- Possibility of human errors during rule application.
- Difficulty in maintaining and modifying hard-coded loan rules.
- Lack of consistent execution of predefined policies.
- Increased manual effort when evaluating customer information.

From an engineering perspective, the project demonstrates the practical application of **compiler design concepts** to a real-world banking automation problem. It integrates language processing, rule validation, code generation, and decision-making into a single system.

1.3 Problem Statement

The engineering problem is to design and implement a Domain-Specific Language compiler for automated loan approval that can process structured banking rules and convert them into executable logic while ensuring syntactic and semantic correctness.

The system must handle multiple interacting factors such as customer age, credit score, annual income, employment status, loan type, and loan amount while evaluating predefined banking policies.

The system must address the technical complexity of processing user-defined rules through lexical, syntactic, and semantic analysis while maintaining accurate and consistent decision-making. It must also operate within realistic constraints such as supporting predefined banking rules, focusing specifically on loan approval, and handling the required customer information.

1.4 Project Aim

The aim of this project is to design and develop a Domain-Specific Language compiler using RegulaScript to automate loan approval by converting structured banking rules into executable logic and evaluating customer information to generate accurate, consistent, and transparent loan decisions.

1.5 Project Objectives

1. To design a Domain-Specific Language called RegulaScript for representing loan approval rules.

2. To develop a lexical analyzer for identifying keywords, identifiers, operators, and literals from RegulaScript rules.
3. To implement syntax and semantic analysis to validate the structure, field names, data types, and conditions of loan rules.
4. To generate and optimize intermediate code and produce executable compiled loan rules.
5. To implement a Loan Compliance and Loan Decision Engine for evaluating customer information against compiled banking policies.
6. To test the compiler and decision engine using different customer information and predefined loan approval rules.
7. To evaluate the system based on accurate, consistent, and transparent loan decisions while reducing manual effort and human errors.

1.6 Scope

Included

The project includes:

- Development of the **RegulaScript Domain-Specific Language**.
- Definition of predefined loan approval rules.
- Lexical analysis of RegulaScript input.
- Syntax analysis and parse-tree construction.
- Semantic validation of rules.
- Intermediate code generation.
- Optimization of generated code.
- Final compiled rule generation.
- Evaluation of customer information against compiled rules.
- Generation of **Approved, Rejected, or Review Required** decisions.
- Display of the decision reason and execution status.

Excluded

The current project does not include:

- Other banking services apart from loan approval.
- Rules outside the supported predefined RegulaScript banking rules.

- Complete web-interface implementation.
- User authentication and role-based access control.
- Permanent decision history and audit-log storage.
- Batch processing.
- Simultaneous multi-user processing.

Assumptions

- Loan approval policies are represented using the supported RegulaScript syntax.
- Customer information provided to the system is valid and available for evaluation.
- The decision engine operates using rules successfully processed by the compiler.

Boundary Conditions

The system is restricted to loan approval and operates using the predefined banking rules supported by RegulaScript. The current implementation is primarily designed for compiler-based processing and does not provide the complete web-based functionality described as a future deployment option.

1.7 Expected Engineering Outcome

The expected engineering outcome is a working prototype of a Domain-Specific Language compiler and loan decision system that demonstrates the automation of loan approval using compiler design principles.

The system is expected to:

- Provide RegulaScript for structured loan-rule definition.
- Convert rules into executable logic through a compiler pipeline.
- Validate loan rules for syntactic and semantic correctness.
- Evaluate customer information against compiled banking policies.
- Generate Approved, Rejected, or Review Required outcomes.
- Provide decision reasons and execution status.
- Reduce manual effort and minimize human errors.
- Provide a scalable foundation for future banking automati

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review was conducted to identify existing approaches for automated loan approval, rule-based decision systems, expert systems, Domain-Specific Languages (DSLs), and business-rule management. Relevant research papers, established decision-modeling standards, and existing rule-engine technologies were considered.

The review focused on identifying how loan eligibility and credit decisions are automated, how business rules are represented and executed, and how transparency and consistency can be achieved. Research publications from areas such as banking automation, expert systems, credit decision-making, and explainable decision support were examined. In addition, Decision Model and Notation (DMN) and rule-engine technologies such as Drools were reviewed as established approaches for representing and executing business rules.

2.2 Review of Existing Work

2.2.1 Rule-Based Expert Systems for Loan Approval

An early example of automated loan decision-making is S.E.A.C., an Expert System for Loan Granting, which was developed to support bank loan risk analysis and granting. The system used an inference engine and multi-stage deduction controlled by meta-rules. It also considered uncertainty through different rule types. This work demonstrated that knowledge-based systems could assist financial analysts in evaluating loan applications.

However, such expert systems are generally dependent on predefined knowledge and inference mechanisms. They do not necessarily provide a modern compiler-based mechanism through which users can define rules in a dedicated language.

2.2.2 Knowledge-Based Loan Underwriting

Kumra, Stein, and Assersohn investigated a knowledge-based approach to commercial loan underwriting. Their work examined decision-support tools for commercial underwriting and

described an expert-system approach that had been used commercially in financial institutions. The study showed the applicability of knowledge-based systems to complex credit-underwriting tasks.

The limitation is that the approach primarily focuses on the underwriting decision process rather than designing a dedicated programming language and compiler pipeline for defining and executing loan rules.

2.2.3 Bank Loan Approval Using Decision Models

Majeske and Lauer studied bank loan approval using a probability-based decision model. Their research considered classification rules and Bayesian decision-making to identify classifications that could optimize expected payoff or minimize expected cost. It also highlighted that different perspectives can result in conflicting loan classifications.

This demonstrates that loan approval may involve multiple interacting factors and conflicting decision objectives. However, statistical decision models differ from the proposed project, which concentrates on explicit banking rules represented through a DSL.

2.2.4 Explainable Loan Underwriting

Research on explainable loan underwriting has investigated belief-rule-base expert systems to automate loan decisions while retaining transparency. Such systems extend traditional IF–THEN rule-based approaches and can represent expert knowledge for lending decisions. The research emphasizes the importance of explaining the decision process rather than producing only an automated prediction.

This is relevant to the proposed system because RegulaScript also uses explicit rules, allowing the system to provide a decision reason along with the final loan decision.

2.2.5 Decision Model and Notation (DMN)

Decision Model and Notation (DMN) is an established standard from the Object Management Group for specifying business decisions and business rules. DMN is designed to make decision logic understandable to both business and technical stakeholders and supports decision tables and executable decision specifications.

DMN demonstrates the importance of representing business decisions in a structured form. However, the proposed project takes a compiler-design-oriented approach, where RegulaScript rules undergo lexical, syntax, semantic, intermediate-code, optimization, and code-generation phases.

2.2.6 Drools Rule Engine

Drools is a business-rule engine capable of storing, processing, and evaluating business rules or DMN decisions. It matches incoming facts with rule conditions and executes applicable rules. Its Drools Rule Language (DRL) provides a textual notation containing rule conditions and actions.

Drools demonstrates how rule engines can separate business rules from application logic. However, the proposed project focuses specifically on demonstrating compiler construction for a loan-approval DSL, rather than using an existing general-purpose rule engine.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
S.E.A.C. Expert System	Uses knowledge-based rules and inference for loan granting; supports multi-stage reasoning and uncertainty.	Older expert-system architecture; dependent on predefined knowledge and inference mechanisms.	Establishes the feasibility of rule-based automated loan decisions.
Knowledge-Based Loan Underwriting	Supports complex commercial underwriting and has practical financial-domain applicability.	Focuses on underwriting decision support rather than a dedicated rule language and compiler.	Supports the use of explicit domain knowledge for loan decisions.
Bayesian Loan Decision Model	Provides mathematical decision-making and	Can involve complex probabilistic modeling and may produce different	Highlights the need to handle multiple

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
	considers expected payoff/cost.	classifications under different perspectives.	interacting loan-approval factors.
Belief-Rule-Based Loan Underwriting	Provides explainable decisions using explicit rules and expert knowledge.	Rule-base design can be complex; approach is not centered on compiler phases.	Supports transparent rule-based loan decisions.
DMN	Standardized representation of business decisions and rules; designed for both business and technical users.	Primarily provides decision modeling and notation rather than a custom compiler specifically for loan approval.	Provides a reference for structured business-rule representation.
Drools Rule Engine	Provides rule storage, matching, and execution; supports business rules and DMN decisions.	Requires an existing rule engine and its rule language; not specifically designed as an educational loan-approval compiler.	Demonstrates practical rule execution and separation of rules from application logic.
Proposed RegulaScript DSL Compiler	Provides a dedicated loan-rule language, compiler pipeline, validation, optimization, and decision generation.	Currently limited to predefined loan rules and loan approval; authentication and complete web deployment are not implemented.	Directly addresses the identified gap by combining compiler design with automated loan approval.

The comparison shows that existing approaches provide strong solutions for rule-based reasoning, probabilistic decision-making, decision modeling, and business-rule execution, but they generally address these aspects separately. The proposed project combines a custom DSL and compiler pipeline with a loan compliance and decision engine.

2.4 Research / Engineering Gap

The review identifies a gap between domain-specific rule representation and the application of compiler-design techniques to automated loan approval.

Existing expert systems and knowledge-based approaches demonstrate that rules can be used for loan evaluation, while DMN and Drools demonstrate structured representation and execution of business decisions. However, these approaches do not specifically focus on developing a compact, dedicated loan-approval DSL with a complete compiler pipeline consisting of lexical analysis, syntax analysis, semantic analysis, intermediate-code generation, optimization, and final code generation.

There is therefore an engineering opportunity to develop a system in which banking rules can be written in a simple domain-specific syntax, automatically validated by compiler phases, transformed into executable logic, and evaluated against customer information.

2.5 Proposed Contribution

The proposed project contributes a Domain-Specific Language called RegulaScript specifically designed for loan approval. Instead of directly hard-coding banking conditions into an application, authorized users can express loan policies using structured RegulaScript rules.

The major contribution is the integration of compiler design and loan decision automation. The Loan Rule Compiler performs lexical analysis, syntax analysis, semantic validation, intermediate-code generation, optimization, and final rule generation. The resulting compiled rules are then supplied to the Loan Compliance and Loan Decision Engine.

The decision engine evaluates parameters such as age, credit score, annual income, employment status, loan type, and loan amount and produces Approved, Rejected, or Review Required outcomes together with the decision reason and execution status.

Thus, the proposed work differs from existing solutions by providing a compiler-based, domain-specific, rule-driven loan approval prototype that aims to improve consistency, transparency, maintainability, and reduce manual effort.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Banking/Loan Officer	Define and modify loan approval rules using a simple structured language.
Authorized Rule Administrator	Create valid RegulaScript rules according to predefined banking policies.
Loan Processing User	Enter customer and loan information for automated evaluation.
Decision Maker	Receive a clear loan decision with the reason for the decision.
System Developer	Maintain and extend the compiler and decision engine.

The system is intended to reduce manual effort and human errors while providing accurate, consistent, and transparent loan decisions.

Functional & Non-Functional Requirements

Requirement	Type (Functional/Non-Functional)	Measurable Target
Accept RegulaScript loan rules	Functional	100% of valid supported rules accepted
Perform lexical analysis	Functional	Correctly identify supported keywords, identifiers, operators and literals
Perform syntax analysis	Functional	Detect invalid rule grammar

Perform semantic analysis	Functional	Validate field names, data types and conditions
Generate executable rules	Functional	Generate executable logic from valid rules
Evaluate customer information	Functional	Process all required customer parameters
Generate loan decision	Functional	Produce Approved, Rejected or Review Required
Provide decision reason	Functional	Display reason for every generated decision
Accuracy	Non-Functional	Consistent results for identical inputs and rules
Reliability	Non-Functional	Avoid crashes during valid rule processing
Maintainability	Non-Functional	Allow banking rules to be modified without redesigning the complete decision engine

The decision engine processes parameters including Customer ID, Age, Credit Score, Annual Income, Employment Status, Loan Type, and Loan Amount.

3.2 Constraints & Applicable Standards

Constraints

The major engineering constraints are:

- The system is designed specifically for **loan approval**.
- Only predefined banking rules supported by RegulaScript are considered.
- The current implementation is primarily console-based.
- Authentication and role-based access control are not implemented.
- Permanent audit-log and decision-history storage is not included.
- Batch processing and simultaneous multi-user processing are outside the current scope.

Applicable Standards / Codes

Standard / Code	Requirement	How Applied in This Project
Compiler design principles	Structured processing of source rules	RegulaScript is processed through lexical, syntax and semantic phases
SQL/database practices	Structured storage of records where database integration is used	SQLite/MySQL may be used for loan records
Web technology standards	Browser-based deployment where applicable	HTML, CSS and JavaScript are specified for future web deployment

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Rule language	RegulaScript	Language	High
Rule validation	Lexical, syntax and semantic validation	Validation stages	High
Customer parameters	Age, credit score, income, employment status, loan type, loan amount	Various	High
Decision output	Approved / Rejected / Review Required	Status	High
Decision explanation	Reason associated with decision	Text	High
Compiler implementation	C	Programming language	High
Development environment	Visual Studio Code + GCC/MinGW	Software	Medium
Database	SQLite/MySQL where required	Database	Medium

Deployment	Web-based deployment as future option	Platform	Medium
------------	--	----------	--------

3.4 Alternative Solutions & Evaluation

Alternative 1: Manual Loan Evaluation

In this approach, loan officers manually compare customer information against banking policies. It is simple to implement but requires significant human effort and may introduce inconsistencies or human errors.

Alternative 2: Machine-Learning-Based Loan Prediction

A machine-learning approach can learn patterns from historical loan data and predict loan approval. However, it requires suitable training data and may make the underlying decision logic less explicit.

Alternative 3: RegulaScript DSL Compiler

The proposed approach allows banking policies to be expressed using a dedicated DSL. The compiler validates and converts these rules into executable logic, which is then evaluated by the loan decision engine.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	25%	2/5	4/5	4/5
Cost	15%	3/5	2/5	4/5
Safety	20%	2/5	3/5	4/5
Sustainability	15%	2/5	3/5	4/5
Reliability	25%	2/5	3/5	5/5

3.5 Selected Design & Detailed Design

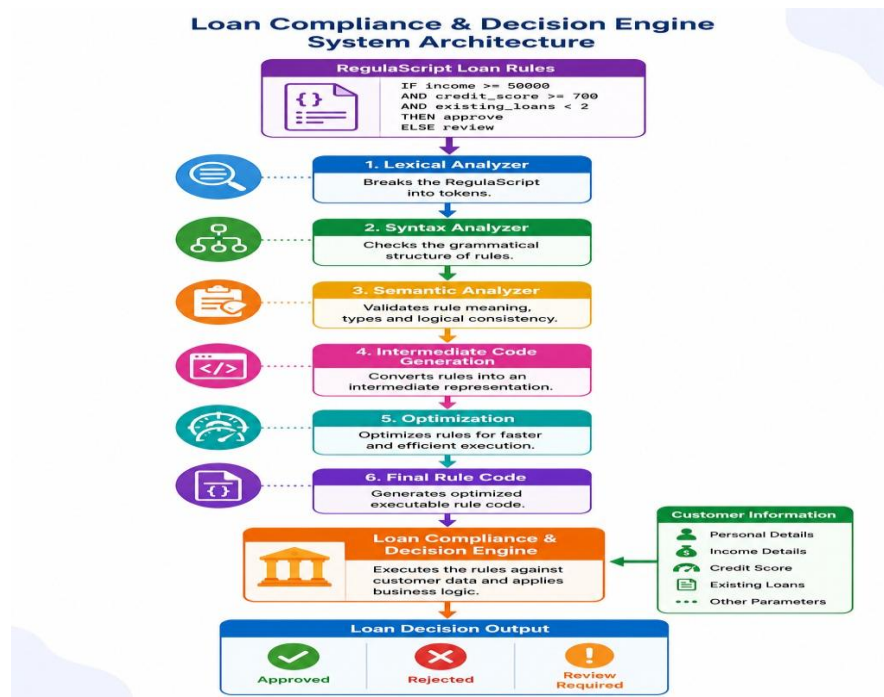
Selected Design

The selected architecture consists of two major modules:

1. **Loan Rule Compiler**
2. **Loan Compliance & Loan Decision Engine**

The Loan Rule Compiler converts RegulaScript rules into executable, machine-readable logic. It performs lexical analysis, syntax analysis, semantic analysis, intermediate-code generation, optimization, and final code generation.

System Architecture



Figurw 3.1 system architecture

Key Engineering Calculation

For a rule-based system, a basic decision can be represented as:

$$\text{Eligibility} = \text{Rule}_1 \text{ AND } \text{Rule}_2 \text{ AND } \dots \text{ AND } \text{Rule}_n$$

For example, if a loan policy contains conditions on credit score, income, employment status, and loan amount, the decision engine evaluates each condition and combines the results according to the defined RegulaScript rule.

Systems Approach

The **Loan Rule Compiler** acts as the rule-processing subsystem and produces validated executable rules. These compiled rules are transferred to the **Loan Compliance and Decision Engine**. The decision engine combines the compiled rules with customer information and evaluates compliance. The final decision and its reason are then presented to the user.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Custom DSL	General-purpose programming language	Simple representation of loan rules	Limited to defined domain	Custom DSL selected because it makes banking rules structured and easier to manage.
Compiler-based processing	Direct rule interpretation	Validation and optimization through compiler phases	More implementation effort	Compiler selected because compiler design is the core engineering focus.
Rule-based decision making	Machine-learning prediction	Transparent and predictable decisions	Less adaptive than ML	Rule-based approach selected because banking policies can be explicitly represented.
Console implementation	Complete web application	Easier implementation and testing	Less user-friendly	Console implementation selected for current prototype, with web deployment as a future option.

SQLite/MySQL	No database	Allows structured record storage	Additional implementation complexity	Database retained as an optional integration for loan records.
--------------	-------------	----------------------------------	--------------------------------------	--

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reduce repetitive manual processing and unnecessary resource use.	Automated rule processing and reusable compiled rules.	Supports automation and reduces repetitive manual work.
Ethics	Loan decisions should be consistent and explainable.	Explicit banking rules and decision reasons.	Rule-based design selected to improve transparency.
Health & Safety	The system is software-based and does not directly control physical equipment.	No physical actuation or hazardous hardware is involved.	Physical safety risk is low.
Security	Customer and loan information must be protected from unauthorized access or manipulation.	Current system limitations include the absence of authentication and role-based access control.	Security controls are identified as important future enhancements.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Invalid RegulaScript rule	Medium	Medium	Medium	Lexical, syntax and semantic validation	Low
Incorrect rule definition	Medium	High	High	Validate rules before compilation and testing	Medium
Incorrect customer input	Medium	High	High	Input validation and data-type checking	Medium
Unauthorized access	Medium	High	High	Implement authentication and role-based access control in future versions	Medium

Exposure of loan/customer data	Low	High	Medium	Secure storage and controlled access	Low–Medium
Compiler execution error	Low	High	Medium	Testing and error handling across compiler phases	Low
Incorrect decision logic	Low	High	Medium	Test rules with multiple customer cases	Low
Lack of audit history	Medium	Medium	Medium	Introduce permanent decision logs in future versions	Low–Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project was developed using a modular compiler-based approach in which the RegulaScript loan rules are processed through sequential compiler phases. The development begins with rule definition and lexical analysis, followed by syntax analysis, semantic validation, intermediate-code generation, optimization, and final rule generation. The compiled rules are then integrated with the Loan Compliance and Loan Decision Engine, which evaluates customer information and produces the final loan decision. The implementation uses C as the primary programming language with Visual Studio Code and GCC/MinGW as the development environment.

Development Resources

- Programming Language: **C**
- IDE: **Visual Studio Code**
- Compiler: **GCC / MinGW**
- Operating System: **Windows or Linux**
- Web Technologies: **HTML, CSS, JavaScript**
- Database: **SQLite or MySQL**
- Standard C Libraries: **stdio.h, stdlib.h, string.h, ctype.h**

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

The project is a software-based compiler and decision-support system, so no specialized hardware or fabrication process is required. The implementation consists of the RegulaScript compiler pipeline and the Loan Compliance and Loan Decision Engine. The compiler converts loan rules into executable logic, while the decision engine evaluates customer information against the compiled rules.

Tool / Software / Equipment	Purpose	Stage Used
C	Implementation of compiler and decision-processing logic	Development
Visual Studio Code	Source-code development and editing	Development
GCC / MinGW	Compilation and execution of C programs	Implementation & Testing
HTML	Development of web-based user interface	Proposed deployment
CSS	Interface styling	Proposed deployment
JavaScript	Client-side interaction	Proposed deployment
SQLite / MySQL	Storage of loan records	Proposed database integration
Standard C Libraries	String, character, input/output and memory-related operations	Compiler implementation

Implementation Evidence

Insert here:

- RegulaScript source-code screenshot
- Lexical analyzer output screenshot
- Syntax/semantic analysis output
- Intermediate-code generation screenshot
- Final compiled rule screenshot
- Loan Decision Engine output screenshot

4.3 Test Plan & Verification

Testing is performed to verify that the RegulaScript compiler correctly processes loan rules and that the decision engine generates the appropriate decision based on customer information.

Requirement		Test Method	Acceptance Criterion
RegulaScript input		Provide valid loan rules	Rules should be accepted
Lexical analysis		Test keywords, identifiers, operators and literals	Tokens should be correctly identified
Syntax analysis		Provide valid and invalid rule structures	Invalid syntax should be detected
Semantic analysis		Test valid and invalid fields/data types	Semantic errors should be detected
Intermediate-code generation		Compile valid rules	Intermediate representation should be generated
Code optimization		Process generated intermediate code	Redundant operations should be removed where applicable
Final rule generation		Compile validated rules	Executable rule should be produced
Customer data processing		Provide required customer details	Input should be processed correctly
Loan decision		Evaluate customer data against compiled rules	Correct decision category should be generated
Decision explanation		Test approved/rejected/review cases	Decision reason should be displayed
Specification	Required Value	Achieved Value	Status (Pass/Fail)
RegulaScript processing	Valid rules processed	To be filled from test execution	Pass/Fail
Lexical analysis	Correct token identification	To be filled	Pass/Fail
Syntax analysis	Invalid syntax detected	To be filled	Pass/Fail
Semantic analysis	Invalid fields/types detected	To be filled	Pass/Fail
Intermediate code	Generated for valid rules	To be filled	Pass/Fail

Optimization	Redundant operations removed	To be filled	Pass/Fail
Final compiled rule	Executable logic generated	To be filled	Pass/Fail
Customer information	Required fields accepted	To be filled	Pass/Fail
Loan decision	Three supported outcomes	To be filled	Pass/Fail
Decision reason	Reason displayed	To be filled	Pass/Fail

4.4 Results

The implemented system demonstrates the application of compiler design principles to automated loan approval. RegulaScript rules are processed through the compiler pipeline and converted into executable loan-approval logic. The generated rules are then supplied to the Loan Compliance and Decision Engine for evaluation.

The decision engine accepts customer details including Customer ID, Age, Credit Score, Annual Income, Employment Status, Loan Type, and Loan Amount. Based on the compiled rules, it generates an appropriate loan decision.

Expected Result Categories

Customer/Rule Evaluation	System Output
Customer satisfies approval conditions	Approved
Customer fails required conditions	Rejected
Conditions require further evaluation	Review Required

The results demonstrate that the system can provide consistent and transparent rule-based loan decisions while reducing manual effort and human errors.

Insert results here:

- Compiler execution screenshots
- Sample RegulaScript input
- Token output

- Parse/syntax output
- Semantic validation output
- Compiled-rule output
- Loan decision results

4.5 Data Analysis & Engineering Judgment

The system was evaluated from the perspective of rule processing, decision consistency, transparency, and implementation feasibility. The compiler architecture separates rule definition from decision execution, allowing loan policies to be represented using RegulaScript and processed systematically.

The use of separate compiler phases provides structured validation of the rules. Lexical analysis identifies the components of the language, syntax analysis checks the structure, and semantic analysis verifies the validity of fields, data types, and conditions. Intermediate-code generation and optimization further process the validated rules before final execution.

From an engineering perspective, the rule-based approach provides predictable results when the same customer information is evaluated against the same rules. It also improves transparency because the decision engine can provide the reason for the generated decision.

However, the system is currently limited to predefined banking rules and loan approval. The PPT also identifies the absence of authentication, permanent audit logs, batch processing, and simultaneous multi-user support.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
To design a DSL for loan approval	RegulaScript was designed for representing loan approval rules.	Fully Achieved
To develop lexical analysis	Compiler identifies keywords, identifiers, operators and literals.	Fully Achieved
To implement syntax and semantic analysis	Syntax and semantic validation are included in the compiler pipeline.	Fully Achieved
To generate and optimize intermediate code	Intermediate-code generation and optimization are included.	Fully Achieved
To implement loan decision processing	Loan Compliance and Loan Decision Engine evaluates customer information.	Fully Achieved
To generate transparent decisions	System produces Approved, Rejected or Review Required with reasons.	Fully Achieved
To implement complete web deployment	Web-based deployment is described, but the PPT states that the current system does not include a complete web interface.	Partially Achieved
To provide authentication and role-based access	Not implemented in the current system.	Not Achieved

4.7 Strengths & Limitations

Strengths

- Provides a dedicated **Domain-Specific Language (RegulaScript)** for loan approval.
- Applies multiple compiler phases to banking rules.
- Reduces repetitive manual rule processing.
- Provides consistent rule-based decisions.
- Supports transparent decision reasons.
- Separates loan-rule compilation from decision execution.

- Demonstrates practical application of compiler design concepts.
- Provides a foundation for future banking automation.

Limitations

- Currently focused only on loan approval.
- Supports only predefined RegulaScript banking rules.
- Complete web interface is not implemented.
- User authentication is not implemented.
- Role-based access control is not implemented.
- Permanent decision history and audit logs are not stored.
- Batch processing is not supported.
- Simultaneous multi-user processing is not supported

4.8 Project Planning, Roles & Budget

Project Planning

The project can be organized into the following development milestones:

Requirement Analysis



Literature Review



DSL Design – RegulaScript



Lexical Analyzer



Syntax & Semantic Analyzer



Intermediate Code & Optimization



Loan Decision Engine



Integration



Testing & Verification



Results & Documentation

Suggested Milestone Timeline

Phase	Activity
Phase 1	Requirement analysis and problem identification
Phase 2	Literature review and existing-system analysis
Phase 3	RegulaScript language and grammar design
Phase 4	Loan Rule Compiler implementation
Phase 5	Loan Compliance and Decision Engine implementation
Phase 6	Module integration
Phase 7	Testing and verification
Phase 8	Results analysis and documentation

Student	Assigned Responsibility	Actual Contribution
Juhi Surin	Compiler and system development	RegulaScript/compiler module development and testing
Aniksha Evanjalin	Decision engine, integration and documentation	Loan decision module, system integration, testing and report preparation
Item	Estimated Cost	Actual Cost
Visual Studio Code	₹0	₹0
GCC / MinGW	₹0	₹0
C Standard Libraries	₹0	₹0
HTML / CSS / JavaScript	₹0	₹0
SQLite	₹0	₹0
MySQL Community Edition	₹0	₹0
Existing Computer/Laptop	Existing resource	Existing resource
Total Additional Cost	₹0	₹0

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project “**DSL Compiler for Loan Approval**” addresses the engineering problem of automating loan approval based on predefined banking policies. The project developed **RegulaScript**, a Domain-Specific Language that enables authorized users to define loan approval rules in a simple and structured manner. These rules are processed through compiler phases including lexical analysis, syntax analysis, semantic analysis, intermediate-code generation, optimization, and final code generation.

The compiled rules are evaluated by the **Loan Compliance and Loan Decision Engine** using customer information such as age, credit score, annual income, employment status, loan type, and loan amount. The system generates **Approved, Rejected, or Review Required** outcomes along with the decision reason and execution status. This provides a structured approach to loan evaluation while reducing manual effort and human errors.

Overall, the project demonstrates how compiler design concepts can be applied to a practical banking automation problem. The combination of a domain-specific rule language, compiler pipeline, and decision engine provides a foundation for accurate, consistent, and transparent loan decision-making.

5.2 Future Work

The following enhancements can be considered for future development:

1. **Complete Web Interface:** Develop and deploy a complete web-based interface through which users can enter customer and loan details and receive decisions.
2. **User Authentication:** Implement secure user authentication to prevent unauthorized access to the loan approval system.
3. **Role-Based Access Control:** Introduce different access levels for administrators, rule developers, and loan-processing users.
4. **Decision History and Audit Logs:** Permanently store loan decisions, decision reasons, and execution history for auditing and future reference.
5. **Batch Processing:** Extend the system to process multiple loan applications simultaneously.

6. **Multi-User Support:** Enable multiple authorized users to access and use the system simultaneously.
7. **Expanded Banking Rules:** Extend RegulaScript to support a wider range of banking policies and more complex loan conditions.
8. **Cloud Deployment:** Deploy the system on cloud platforms such as **AWS, Azure, or Heroku** for improved accessibility.

These future enhancements directly address the limitations identified in the current system, including the absence of complete web functionality, authentication, role-based access control, permanent audit logs, batch processing, and multi-user support.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Learned how **Domain-Specific Languages (DSLs)** can be designed for a specific application domain.
- Gained knowledge of compiler phases including **lexical analysis, syntax analysis, semantic analysis, intermediate-code generation, optimization, and code generation**.
- Learned how structured banking rules can be converted into executable decision logic.
- Understood the integration between a compiler and a rule-based decision engine.
- Gained knowledge about applying compiler design concepts to a real-world banking application.

Engineering & Professional Skills Developed

- Compiler design and implementation skills.
- C programming and debugging skills.
- Rule-based system design.
- System architecture and modular development.
- Software testing and verification.
- Problem-solving and engineering decision-making.
- Technical documentation and report preparation.
- Teamwork and project coordination.
- Presentation and communication skills.

Individual Reflection – Juhi Surin

- **Most difficult engineering problem encountered and how it was solved:** Understanding how the RegulaScript rules could be systematically converted into executable logic through different compiler phases was challenging. This was addressed by dividing the compiler into modular phases and validating the rules at each stage.
- **A key engineering decision made personally and the evidence behind it:** Using a dedicated DSL for loan rules was an important decision because it allows banking policies to be represented in a structured and domain-specific manner.
- **New knowledge acquired independently:** Gained practical understanding of lexical analysis, syntax analysis, semantic validation, and rule-based decision processing.
- **What would be redesigned if repeated:** The system could be redesigned with stronger error handling and a more user-friendly interface from the beginning.

Individual Reflection – Aniksha Evanjalin

- **Most difficult engineering problem encountered and how it was solved:** Integrating the compiled loan rules with customer information and producing an appropriate decision was challenging. It was addressed by separating the compiler and decision-engine responsibilities into independent modules.
- **A key engineering decision made personally and the evidence behind it:** Choosing a rule-based decision approach helped maintain transparent and predictable loan decisions because the conditions are explicitly defined in RegulaScript.
- **New knowledge acquired independently:** Learned how compiler concepts can be applied to a practical banking automation system and gained experience in system integration, testing, and technical documentation.
- **What would be redesigned if repeated:** The project could be redesigned with a complete web interface, authentication, and permanent decision-history storage to make the system more practical for real-world deployment.

REFERENCES

- [1] G. Babaei, P. Giudici, and E. Raffinetti, “Explainable FinTech lending,” *Journal of Economics and Business*, vols. 125–126, Art. no. 106126, 2023, doi: 10.1016/j.jeconbus.2023.106126.
- [2] D. Chen, J. Ye, and W. Ye, “Interpretable selective learning in credit risk,” *Research in International Business and Finance*, vol. 65, Art. no. 101940, 2023, doi: 10.1016/j.ribaf.2023.101940.
- [3] L. O. Hjelkrem and P. E. de Lange, “Explaining deep learning models for credit scoring with SHAP: A case study using open banking data,” *Journal of Risk and Financial Management*, vol. 16, no. 4, Art. no. 221, 2023, doi: 10.3390/jrfm16040221.
- [4] J. Dessain, N. Bentaleb, and F. Vinas, “Cost of explainability in AI: An example with credit scoring models,” in *World Conference on Explainable Artificial Intelligence (xAI 2023)*, Communications in Computer and Information Science, vol. 1901, pp. 498–516, 2023, doi: 10.1007/978-3-031-44064-9_26.
- [5] R. Hashimoto, K. Miura, and Y. Yoshizaki, “Application of machine learning to a credit rating classification model: Techniques for improving the explainability of machine learning,” *Bank of Japan Working Paper Series*, 2023.
- [6] A. Khan and S. K. Ghosh, “Machine assistance for credit approval? Random wheel can recommend and explain,” *Expert Systems with Applications*, vol. 215, Art. no. 119231, 2023, doi: 10.1016/j.eswa.2022.119231.
- [7] A. Aljadani, B. Alharthi, M. A. Farsi, H. M. Balaha, M. Badawy, and M. A. Elhosseini, “Mathematical modeling and analysis of credit scoring using the LIME explainer: A comprehensive approach,” *Mathematics*, vol. 11, no. 19, Art. no. 4055, 2023, doi: 10.3390/math11194055.
- [8] S. Agarwal, C. B. Muckley, and P. Neelakantan, “Countering racial discrimination in algorithmic lending: A case for model-agnostic interpretation methods,” *Economics Letters*, vol. 226, 2023.

- [9] M. El Annas, B. Benyacoub, and M. Ouzineb, “Semi-supervised adapted HMMs for P2P credit scoring systems with reject inference,” *Computational Statistics*, vol. 38, no. 1, pp. 149–169, 2023, doi: 10.1007/s00180-022-01220-9.
- [10] D. Bork, S. J. Ali, and G. M. Dinev, “AI-enhanced hybrid decision management,” *Business & Information Systems Engineering*, vol. 65, pp. 17

Implementation output:

The screenshot displays the RegulaScript DSL Compiler interface. The top bar shows the application name 'RegulaScript' and a status 'Engine ready - rules compiled'. The interface is divided into two main sections: Module 1 (Loan Rule Compiler) and Module 2 (Loan Compliance & Decision Engine).

Module 1: Loan Rule Compiler

Write banking rules in **RegulaScript**, then run them through the six compiler phases: Lexical → Syntax → Semantic → Intermediate Code → Optimizer → Target Code.

```
RULE R001 IF CreditScore >= 750 AND AnnualIncome > 600000 AND Age >= 21 AND Age <= 58 THEN APPROVE;  
RULE R002 IF CreditScore < 550 THEN REJECT;  
RULE R003 IF EmploymentStatus == "UNEMPLOYED" THEN REJECT;  
RULE R004 IF CreditScore >= 350 AND CreditScore < 750 AND AnnualIncome >= 300000 THEN REVIEW;  
RULE R005 IF LoanAmount > 5000000 THEN REVIEW;
```

Compile Rules Compiled 5 rule(s) successfully.

rule set name (e.g. demo_v1) **Save to File** Load saved rule set...

Tokens Parse Tree Semantic Intermediate Code Optimized Code Target Code

66 tokens

```
RULE 'RULE' (line 1)  
IDENT 'R001' (line 1)  
IF 'IF' (line 1)  
IDENT 'CreditScore' (line 1)  
GE '>=' (line 1)  
NUMBER '750' (line 1)  
AND 'AND' (line 1)  
IDENT 'AnnualIncome' (line 1)  
GT '>' (line 1)  
NUMBER '600000' (line 1)  
AND 'AND' (line 1)  
IDENT 'Age' (line 1)  
GE '>=' (line 1)  
NUMBER '21' (line 1)
```

Module 2: Loan Compliance & Decision Engine

Enter a customer profile. The engine runs it against the compiled rule bytecode from Module 1 and returns the first matching decision.

Customer ID: C1001 Customer Name: Juhi Sakshi Surin

Age: 29 Credit Score: 780

Annual Income (₹): 750000 Loan Amount (₹): 1200000

Employment Status: EMPLOYED Account Type: SAVINGS

Loan Type: PERSONAL

Evaluate Loan Decision Decision generated.

Approved

Eligibility Status: Eligible

Figure a : Implementation output 1

The screenshot displays the RegulaScript DSL Compiler interface, showing the detailed tokenization of the rules and the decision output.

Module 1: Loan Rule Compiler

Write banking rules in **RegulaScript**, then run them through the six compiler phases: Lexical → Syntax → Semantic → Intermediate Code → Optimizer → Target Code.

```
RULE R001 IF CreditScore >= 750 AND AnnualIncome > 600000 AND Age >= 21 AND Age <= 58 THEN APPROVE;  
RULE R002 IF CreditScore < 550 THEN REJECT;  
RULE R003 IF EmploymentStatus == "UNEMPLOYED" THEN REJECT;  
RULE R004 IF CreditScore >= 350 AND CreditScore < 750 AND AnnualIncome >= 300000 THEN REVIEW;  
RULE R005 IF LoanAmount > 5000000 THEN REVIEW;
```

Compile Rules Compiled 5 rule(s) successfully.

rule set name (e.g. demo_v1) **Save to File** Load saved rule set...

Tokens Parse Tree Semantic Intermediate Code Optimized Code Target Code

66 tokens

```
RULE 'RULE' (line 1)  
IDENT 'R001' (line 1)  
IF 'IF' (line 1)  
IDENT 'CreditScore' (line 1)  
GE '>=' (line 1)  
NUMBER '750' (line 1)  
AND 'AND' (line 1)  
IDENT 'AnnualIncome' (line 1)  
GT '>' (line 1)  
NUMBER '600000' (line 1)  
AND 'AND' (line 1)  
IDENT 'Age' (line 1)  
GE '>=' (line 1)  
NUMBER '21' (line 1)
```

Module 2: Loan Compliance & Decision Engine

Enter a customer profile. The engine runs it against the compiled rule bytecode from Module 1 and returns the first matching decision.

Customer ID: C1001 Customer Name: Juhi Sakshi Surin

Age: 29 Credit Score: 780

Annual Income (₹): 750000 Loan Amount (₹): 1200000

Employment Status: EMPLOYED Account Type: SAVINGS

Loan Type: PERSONAL

Evaluate Loan Decision Decision generated.

Approved

Eligibility Status: Eligible

Matched Rule: R001

Decision Reason: Customer C1001 matched rule R001 -> APPROVE

Execution Status: SUCCESS

Figure b : Implementation output 2

Procedure Description

1. Open the RegulaScript DSL Compiler application.
2. Enter the loan rules in the Rule Compiler text area using the RegulaScript syntax.
3. Click “Compile Rules” to start the compilation process.
4. The rules pass through the six compiler phases:
5. Lexical Analysis
6. Syntax Analysis
7. Semantic Analysis
8. Intermediate Code Generation
9. Code Optimization
10. Target Code Generation
11. The Tokens tab displays the tokens generated from the entered rules, such as RULE, IDENT, IF, GE, NUMBER, and AND.
12. The Parse Tree and Semantic tabs can be used to verify the syntactic and semantic correctness of the rules.
13. The Intermediate Code, Optimized Code, and Target Code tabs display the corresponding outputs of the remaining compiler phases.
14. After successful compilation, enter the customer details such as Customer ID, Age, Credit Score, Annual Income, Loan Amount, Employment Status, Account Type, and Loan Type.
15. Click “Evaluate Loan Decision” to execute the compiled rules against the customer profile.
16. The decision engine checks the rules in order and identifies the first matching rule.
17. For the given customer, Rule R001 matches because the credit score, annual income, age, and other conditions satisfy the rule.
18. The system generates the final decision as “Approved”, with the eligibility status shown as “Eligible” and execution status as “SUCCESS”.

APPENDICES

Appendix A – Detailed Calculations

This appendix contains the detailed calculations and rule-evaluation steps used in the loan approval decision process.

The RegulaScript rules are processed through the compiler stages and the resulting conditions are evaluated against customer information such as:

- Customer ID
- Age
- Credit Score
- Annual Income
- Employment Status
- Loan Type
- Loan Amount

The calculated/evaluated conditions are used by the Loan Compliance & Loan Decision Engine to generate one of the following outcomes:

- **Approved**
- **Rejected**
- **Review Required**

The detailed calculations should be included only where they are available from the actual implementation and testing.

Appendix B – Engineering Drawings / Schematics

B.1 Overall System Architecture

The system architecture consists of the following major stages:

RegulaScript Loan Rules → Lexical Analyzer → Syntax Analyzer → Semantic Analyzer → Intermediate Code Generation → Optimization → Final Rule Code → Loan Compliance & Loan Decision Engine → Loan Decision Output

B.2 Compiler Pipeline

The compiler pipeline consists of:

1. RegulaScript Rule Input
2. Lexical Analysis
3. Syntax Analysis
4. Semantic Analysis
5. Intermediate Code Generation
6. Optimization
7. Final Compiled Rule

B.3 Loan Decision Flow

Customer Information → Compiled Rule Evaluation → Compliance Checking → Decision Generation → Approved / Rejected / Review Required

The architecture diagram and flowchart used in the main report may be included here in higher resolution if required.

Appendix C – Source Code / Code Repository Information

The project is implemented using C and developed using **Visual Studio Code** with **GCC/MinGW** for compilation.

The implementation includes the major compiler components required for processing the RegulaScript language:

- Lexical analysis
- Syntax analysis
- Semantic analysis
- Intermediate code generation
- Code optimization
- Final rule generation
- Loan compliance and decision evaluation

C.1 Essential Code Excerpts

Only essential code excerpts should be included in the printed report.

These may include:

- Token identification
- Keyword and identifier recognition
- Operator recognition
- Syntax validation
- Semantic checking
- Rule evaluation
- Loan decision generation

C.2 Complete Source Code

The complete source code may be maintained separately in the approved project repository, as permitted by the institution.

Repository: To be provided by the project team, if applicable.

Appendix D – Datasheets

No external hardware or electronic components are specified for the current DSL Compiler for Loan Approval project.

Therefore, no hardware datasheets are applicable.

The project primarily uses software tools and technologies including:

- C
- Visual Studio Code
- GCC/MinGW
- Windows/Linux
- HTML
- CSS
- JavaScript
- SQLite/MySQL, where applicable

Appendix E – Experimental / Raw Data

This appendix contains the test inputs and corresponding outputs used to verify the loan approval decision engine.

E.1 Sample Customer Input Parameters

Parameter	Description
Customer ID	Unique customer identification
Age	Customer age
Credit Score	Customer credit score
Annual Income	Customer annual income
Employment Status	Employment information
Loan Type	Type of loan requested
Loan Amount	Requested loan amount

E.2 Decision Outputs

The decision engine produces one of the following results:

Output	Meaning
Approved	Customer satisfies the defined loan rules
Rejected	Customer does not satisfy the defined loan rules
Review Required	Application requires further evaluation

Appendix F – Ethics / Institutional Approval

No separate ethics or institutional approval is specified in the project materials.

The project is a software-based compiler and loan decision demonstration system. It does not indicate the use of human-subject experiments or collection of personal customer data for research purposes.

If the institution requires an approval/declaration for the project, the relevant document should be attached here.

Appendix G – Project Meeting / Progress Records

This appendix may contain evidence of project development and supervision, including:

- Project meeting records
- Discussions with the project supervisor
- Progress review records
- Project milestones
- Screenshots of project development
- Review and feedback records
- Presentation/progress evaluation documents

G.1 Project Team

Project Title: DSL Compiler for Loan Approval

Team Members:

1. Juhi Surin – 192525148
2. Aniksha Evanjalín – 192521308

Project Supervisor: Dr. N. Deepa

Actual meeting records and screenshots should be attached if available.

Appendix H – User/Industry Feedback

No specific user or industry feedback is provided in the project materials.

If feedback was collected during project evaluation or demonstration, the following evidence may be included:

- Faculty feedback
- User evaluation forms
- Demonstration feedback
- Suggestions received during project review
- Evaluation comments

If no formal feedback was collected, this appendix may be marked:

“Not Applicable – No formal user/industry feedback was collected for the project.”

Appendix I – Publications / Patents / Competitions / Product Outcomes

No publications, patents, competitions, or commercial product outcomes are specified in the available project materials.

Therefore:

“Not Applicable – No publication, patent, competition participation, or commercial product outcome is reported for this project.”

Appendix J – Individual Contribution Evidence

This appendix is **mandatory because the project is a team project**.

J.1 Team Member 1 – Juhi Surin

Register Number: 192525148

Evidence of individual contribution may include:

- Contribution to project planning
- Contribution to RegulaScript design
- Contribution to compiler development
- Contribution to testing and validation
- Contribution to documentation
- Contribution to project presentation

Supporting evidence may include relevant source-code sections, screenshots, Git/repository commits, development records, or review documents.

J.2 Team Member 2 – Aniksha Evanjalini

Register Number: 192521308

Evidence of individual contribution may include:

- Contribution to project planning
- Contribution to RegulaScript design
- Contribution to compiler development
- Contribution to testing and validation
- Contribution to documentation
- Contribution to project presentation

Supporting evidence may include relevant source-code sections, screenshots, Git/repository commits, development records, or review documents.

J.3 Contribution Evidence Table

Team Member	Evidence to Attach
Juhi Surin (192525148)	Source-code contribution, screenshots, repository records, documentation, presentation/progress evidence
Aniksha Evanjalin (192521308)	Source-code contribution, screenshots, repository records, documentation, presentation/progress evidence

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)

Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)